

Lab 3

Zagadnienia do opanowania:

- Obsługa zegara
 - Interfejs użytkownika
 - Komponent spline
-

Zadania:

1. Akcja: skok

- 1.1. Dodajemy nowy komponent Input Action o nazwie IA_Jump. Tutaj nic nie zmieniamy, czyli zostaje typ (Value Type) digital. W obiekcie Lab1IMC dodajemy nową akcję i podpinamy pod nią klawisz SpaceBar.
- 1.2. Przechodzimy do Lab1Character i w EventGraphs dodajemy obsługę zdarzenia IA_Jump podpinając funkcję Jump.

2. Utworzenie zegara liczącego czas gry.

- 2.1. Tworzymy nowy katalog w Content o nazwie HUD, a następnie w tym katalogu tworzymy obiekt Widget Blueprint o nazwie TimerHUD (PPM->UserInterface->WidgetBlueprint). Edytujemy ten blueprint.
- 2.2. Dodajemy do naszego komponentu najpierw obiekt typu CanvasPanel (sekcja Panel), następnie do Panelu dodajemy HorizontalBox, a następnie dwa komponenty typu Text (ostatnia opcja w sekcji Common). W pierwszym, Text ustawiamy na „Timer:”. W drugim ustawiamy nazwę TimerTB. Następnie w oknie z lewej strony zaznaczmy HorizontalBox i ustawiamy go na środku przy górnej krawędzi. W tym celu w ustawieniach (Details) klikamy na Combo Anchors (na samej górze) i wybieramy kwadrat przy górnej krawędzi na środku. Ustawiamy jego szerokość (dostosować w zależności od rozmiaru czcionki) 200x55 i ustawić Alignment na (0,5; 0).
- 2.3. Wybieramy TimerTB i dodajemy wiązanie (binding) dla parametru Text – dostaniemy nową funkcję. Pewnie domyślnie będzie się ona nazywała GetText. Dodajemy nową zmienną typu float o nazwie Seconds, dodajemy do funkcji (przeciągamy i wybieramy GetSeconds) i wywołujemy funkcję TimeSecondsToString. Wynik tej funkcji podpinamy do ReturnNode.
- 2.4. Otwieramy LevelBlueprint. Wywołujemy funkcję CreateWidget. Do OwningPlayer podpinamy wyjście funkcji GetPlayerController, a w Combo Class wybieramy nasz TimerHUD. Następnie na pinie ReturnValue klikamy PPM i wybieramy opcję promoteToVariable i nazywamy tę zmienną TimerHUD. Następnie przeciągamy z Set ustawiającego TimerHUD niebieską linię i wywołujemy funkcję AddToViewport.
- 2.5. Dalej wywołujemy funkcję SetTimerByFunctionName ustawiamy Looping na true i Time na 0,1. Następnie z ReturnValue klikamy PPM i wybieramy Promote To Variable nazywamy tę zmienną TimerForHUD (to jest handler timera – nie mylić z TimerHUD, który jest referencją do Widgetu). Nazwę funkcji w SetTimer... ustawimy po tym jak tę funkcję utworzymy.
- 2.6. Tworzymy funkcję o nazwie UpdateTimer. Dodajemy zmienną SecondsElapsed (float) i ustawiamy ją jako sumę: SecondsElapsed i wartości zwracanej przez funkcję GetTimerElapsedTimeByHandle wywołanej dla TimerForHUD. Następnie, dla widgetu TimerHUD ustawiamy wartość zmiennej Seconds na SecondsElapsed.
- 2.7. Modyfikujemy całość, bo nowy zegar nie przestaje działać po zakończeniu rozgrywki. Dlatego dodajemy zmienną bool IsGameFinished, którą ustawiamy na false. Następnie,

przy wykryciu zakończenia gry (OnActorBeginOverlap (TriggerBox)) przed Delay ustawiamy IsGameFinished na true. Następnie modyfikujemy funkcję UpdateTimer sprawdzając w pierwszej kolejności, czy wartość IsGameFinished jest False (komponent Branch).

3. MenuStart

- 3.1. W katalogu HUD dodajemy nowy Widget o nazwie MenuStartHUD. Dodajemy do niego CanvasPanel. Do Canvas dodajemy VerticalBox, a do niego: Button, Border i znowu Button. W Border ustawiamy padding na 20 (ma on pełnić tylko rolę odstępu między Buttonami), a kolor Brush->Tint i BrushColor zmieniamy Alpha koloru na 0 (żeby był przezroczysty). Następnie dla VerticalBox Anchor ustawiamy na środek ekranu (jeśli trzeba zmodyfikować należy Alignment, Position, Size, tak, żeby cały komponent był na środku ekranu).
- 3.2. Nazywamy Buttony: górny: StartButton a dolny QuitButton. Do Buttonów dodajemy text Block i wpisujemy odpowiednio StartGame i Quit. Dostosowujemy rozmiar VerticalBox do długości napisów.
- 3.3. Wracamy do edycji LevelBlueprint. To co w tej chwili jest podpięte pod BeginPlay powinno być wywołane dopiero po rozpoczęciu rozgrywki. Podpinamy to wszystko pod nowe zdarzenie StartGame. PPM->AddCustomEvent. Do BeginPlay podpinamy CreateWidget dla MenuStartHUD. Zapamiętujemy MenuStartHUD w zmiennej (Promote to Variable) StartMenuHUD. Dalej wyłączamy możliwość sterowania ludzikiem: SetInputModeUIOnly i następnie wywołujemy SetShowMouseCursor na true (tutaj należy wyłączyć ContextSensitive dla funkcji).
- 3.4. To co zostało ustawione w BeginPlay trzeba na początku uruchomienia StartGame wycofać. Dlatego wywołujemy SetInputModeGameAndUI, następnie SetShowMouseCursor na false, a następnie usuwamy (RemoveFromParent) zapamiętaną zmienną StartMenuHUD. Reszta funkcji w StartGame zostaje.
- 3.5. Wracamy do MenuStartHUD. Dla obu przycisków zaznaczamy IsVariable (zaraz obok nazwy) i następnie w zdarzeniach (Events) klikamy na OnClicked. Pojawia się w sekcji Graph nowe zdarzenia. Dla zdarzenia Quit sprawa jest prosta podpinamy funkcję QuitGame. Dla StartGame tworzymy nowy Dispatcher w Game Mode o nazwie OnStartGameDispatcher następnie w zdarzeniu naciśnięcia OnStart w MenuHUD pobieramy GameMode i rzutujemy na Lab1GM. Następnie wywołujemy ten dispatcher (CallOnStartGameDispatcher). Przechodzimy do LevelBlueprint i dodajemy funkcjonalność przy BeginPlay: pobieramy GameMode, rzutujemy na Lab1GM i wywołujemy BindEventToOnStartGameDispatcher. Event z ostatniej funkcji trzeba połączyć ze zdarzeniem StartGame (te czerwone kwadraciki).

4. SplineActor

- 4.1. Tworzymy nowy blueprint typu Actor o nazwie SplineActor. Do komponentów tego aktora dodajemy Cube i Spline (bez hierarchii). Przesuwamy Cube do góry, żeby nie był w podłodze. Tak przygotowanego aktora dodajemy do planszy i edytujemy jego spline (zaznaczmy punkt i z przytrzymanym klawiszem ALT przesuwamy). Następnie dla Spline zaznaczamy Closed Loop.
- 4.2. Przechodzimy do EventGraph. Dodajemy zmienną typu float o nazwie TimeToComplete, która będzie decydować o tym, ile czasu ma zająć aktorowi przejście całej ścieżki. Robimy tę zmienną publiczną. Następnie dodajemy nowy komponent TimeLine (Add Timeline). Zostanie dodana do komponentów zmienna Timeline, dla której ustawiamy SetPlayRate na wartość 1/TimeToComplete. Następnie uruchamiamy Timeline (Play). Ustawiamy jeden track o nazwie SplineTrack o długości 1s i dwóch punktach 0,0 i 1,1. Przy każdym Update

z Timeline będziemy Ustawiać wartość transformacji aktora (funkcja SetActorTransform). Z pinu Transform tej funkcji wyciągamy funkcję MakeTransform. Następnie pobieramy z komponentu Spline GetSplineLength i wywołujemy funkcję Lerp (float) A zostaje puste, pod B podpinamy GetSplineLength, a pod Alpha splineTrack z Timera). Następnie dla Spline pobieramy dwie wartości GetLocationAtDistanceAlongSpline i GetRotationAtDistanceAlongSpline, gdzie Distance jest wynikiem funkcji Lerp. Wyniki obu tych funkcji podpinamy pod MakeTransform.

5. Strzelanie

- 5.1. Dodajemy nowy blueprint typu Actor i nazywamy go Projectile. Otwieramy i dodajemy komponenty: BoxCollision, Cylinder i ProjectileMovement. BoxCollision robimy domyślnym głównym komponentem (łapiemy i przesuwamy nad SceneRoot), a Cylinder robimy dzieckiem Box'a. Dla Boxa ustawiamy następujące parametry: Scale 0,5, 0,3, 0,3. Następnie w sekcji Collision ustawiamy CollisionPreset na Custom i zaznaczamy Block dla wszystkich typów oprócz Pawn dla którego zaznaczamy Ignore. Następnie rozwijamy sekcję Advanced i zaznaczamy use CCD. Zaznaczamy jeszcze Simulation Generates Hit Events. Dalej, dla Cylindera ustawiamy: scale na 0,4, 0,6, i 1,0 oraz Rotation 0, -90, 0. Dla ProjectileMovement ustawiamy: InitialSpeed 3000, maxSpeed 3000, ShouldBounce na true i Velocity na 3000, 0, 0. W EventGraph w obsłudze zdarzenia HIT wywołujemy DestroyActor.
- 5.2. Dodajemy nowe zdarzenie wejściowe IA_Shoot (domyślne) i dodajemy jego obsługę w IMC na lewy klawisz myszy. W Lab1Character w obsłudze zdarzenia IA_Shoot dodajemy SpawnActor i wybieramy klasę Projectile. Do SpawnTransform dodajemy MakeTransform. Dodajemy nową zmienną ProjectileOffset typu Vector i ustawiamy jej wartość na (100, 0, 0). Następnie pobieramy wartości funkcji GetActorLocation i GetActorRotation. Dla ProjectileOffset wywołujemy RotateVector i wynik dodajemy do GetActorLocation. Wynik sumy wektorów podpinamy pod Location z MakeTransform, a wyjście z GetActorRotation pod Rotation z MakeTransform. Testujemy. Żeby ograniczyć liczbę pocisków między pinami Triggered zdarzenia IA_Shoot a SpawnActor dodajemy funkcję DoOnce, którą resetujemy podpinając Completed ze zdarzenia IA_Shoot.